**Task:** This a two-player game is played on a board consists of n x n fields, where each field contains a value between 0 and 4. Initially, all the fields contain the value of 0. If a player chooses a field, then the value of the field and its neighbours incremented by one (if the value is less than 4). The player's score represents how many fields did he make to have the value of 4. If a value of a field reaches 4, then the field is colorized with the color of the actual player (red or blue). The game ends, when all fields have the value of 4. The player having the higher score wins.
Implement this game, and let the board size be selectable (3x3, 5x5, 7x7). The game should recognize if it is ended, and it has to show in a message box which player won. After this, a new game should be started automatically.

**Analysis of the Task**

The program must provide a responsive Swing GUI with dynamic board resizing. Game logic must be separated from the UI. User clicks must be processed, the board updated in real time, and the game must automatically detect completion, announce the winner, and restart. All board sizes must fit perfectly in the window without scrollbars.

**Solution Plan**

1. Define a Board class to manage game state: values, ownership, scores, current player.
2. Define a FourGame class (extends JFrame) for the GUI: JComboBox for size, dynamic JButton grid.
3. Use a single ActionListener to handle both size changes and cell clicks.
4. After each move, update the UI, check for game over, show winner dialog, and restart.
5. Ensure pack() is called so window always fits the board.

## Key Method Descriptions

**Board(int size) //Action handler**

Constructor that creates a fresh n × n board.

- Initialises two parallel 2D arrays: values (all 0) and owners (all -1).
- Stores the board size in the final field n.
- Prepares empty score counters for both players.

**Board.processMove(int x, int y) /// Action event**

Processes a complete player move at the clicked cell (x = row, y = column).

- Increments the clicked cell and its existing orthogonal neighbours (up, down, left, right) by calling the private helper inc().
- Guarantees no out-of-bounds access with boundary checks.
- Forms the classic "plus" pattern of five (or fewer on edges/corners) affected cells.

**Board.inc(int x, int y)** (private) /// Action event

Safely increments a single cell.

- If the current value is less than 4 → value++.
- When the value reaches exactly 4 → assigns ownership to the current player (owners[x][y] = currentPlayer) and awards one point (scores[currentPlayer]++).
- Prevents any value from ever exceeding 4.

**Board.isGameOver() /// Action event**

Checks whether the game has ended.

- Returns true only when every cell in values[][] equals 4.
- Used after each move to decide whether to show the winner dialog.

**Board.getWinner()**

Determines the final result.

- Compares the two players' scores: returns 0 → Red wins, 1 → Blue wins, -1 → Tie.

**FourGame()** (constructor)

Builds the main JFrame window.

- Sets title, BorderLayout, and creates the top panel containing the "Board size:" label and JComboBox ("3x3", "5x5", "7x7"). **/// Action listener**
- Registers itself as the single ActionListener for all events.
- Calls startNewGame(3) to start a default 3×3 game, then pack(), centres the window and makes it visible.

**FourGame.startNewGame(int size)**

Completely rebuilds the board for a new size (called at start-up and whenever the combo box changes).

- Creates a new Board object.
- Removes the previous centre panel (if any).
- Builds a fresh JPanel with GridLayout(size, size) and an n × n array of JButtons (initial text "0", white background).
- Adds the new panel to the centre, revalidates, repaints and calls pack() so the window automatically resizes perfectly to the new grid.

**FourGame.actionPerformed(ActionEvent e)**

Central event router (single ActionListener for the whole application).

- If the source is the size combo box → extracts the first digit ("3x3" → 3) and calls startNewGame(newSize).
- Otherwise loops over the button array to identify which grid button was pressed → calls processClick(row, col) with the correct coordinates.

**FourGame.processClick(int row, int col) ///Action handler**

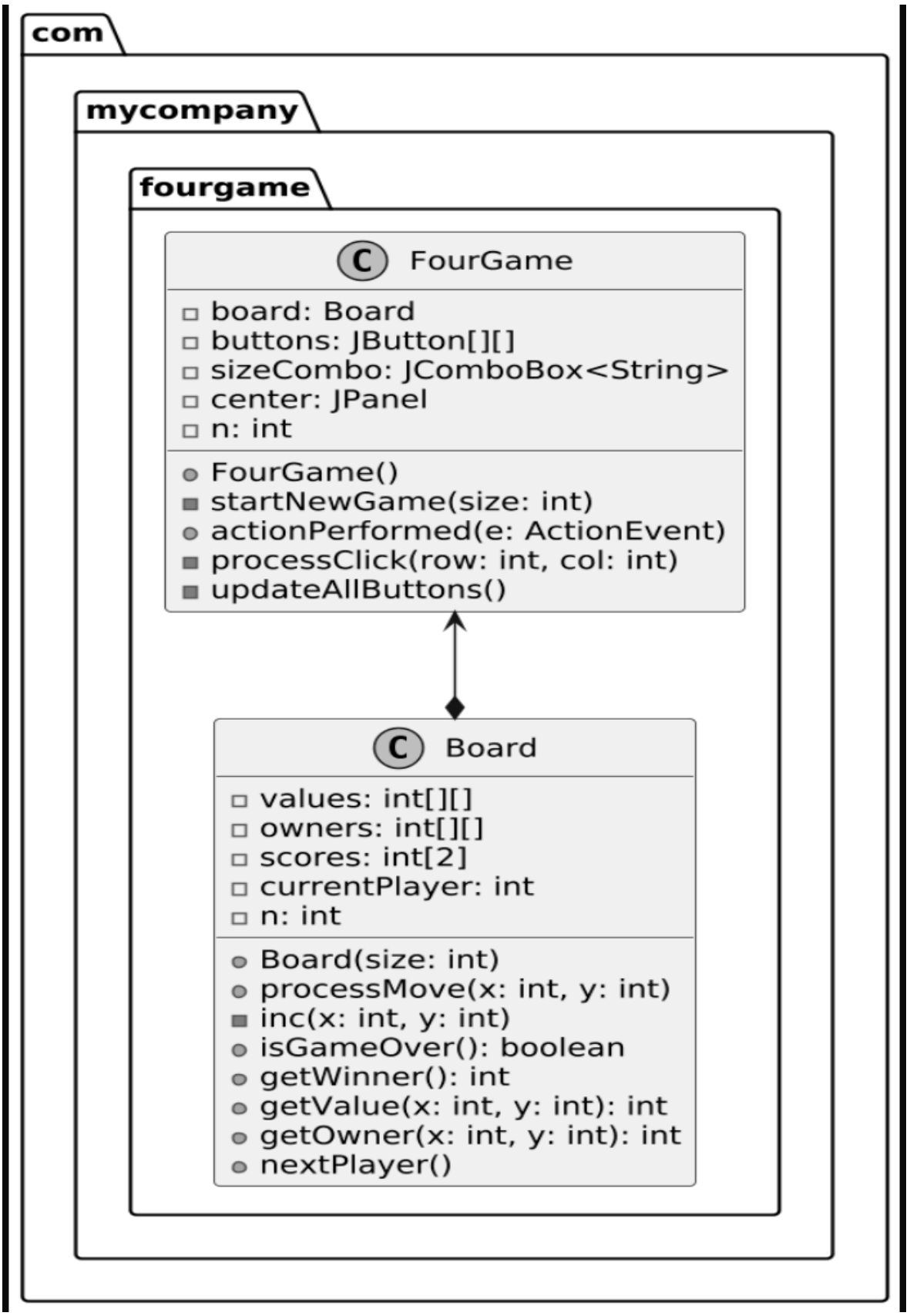
Handles one player click on the visible grid.

- Forwards the move to board.processMove(row, col).
- Refreshes the entire UI with updateAllButtons().
- If board.isGameOver() is true → determines the winner, shows a JOptionPane with “Red wins!”, “Blue wins!” or “Tie!”, then automatically restarts a new game with the same size.
- Otherwise switches the current player with board.nextPlayer().

**FourGame.updateAllButtons()**

Synchronises every JButton with the current state of the Board.

- Sets button text to the current value (0–4).
- Colours the button RED if owner == 0, BLUE if owner == 1, WHITE if still unowned.
- Executed after every move to give immediate visual feedback.


com
mycompany
fourgame
FourGame
board: Board
buttons: JButton[][]
sizeCombo: JComboBox<String>
center: JPanel
n: int
FourGame()
startNewGame(size: int)
actionPerformed(e: ActionEvent)
processClick(row: int, col: int)
updateAllButtons()
Board
values: int[][]
owners: int[][]
scores: int[2]
currentPlayer: int
n: int
Board(size: int)
processMove(x: int, y: int)
inc(x: int, y: int)
isGameOver(): boolean
getWinner(): int
getValue(x: int, y: int): int
getOwner(x: int, y: int): int
nextPlayer()

**User Story Test Cases (15 Edge Cases)**

**1** AS A user I WANT TO change board size GIVEN combo box is visible WHEN I select 5×5 THEN board becomes 5×5 and window resizes.
**2** AS A user I WANT TO play on default board GIVEN program starts WHEN I click a cell THEN it and neighbors increment.
**3** AS A user I WANT TO see ownership GIVEN cell reaches 4 WHEN Red causes it THEN button turns red.
**4** AS A user I WANT TO know winner GIVEN board is full WHEN last move is made THEN dialog shows correct winner.
**5** AS A user I WANT TO restart automatically GIVEN winner dialog WHEN I click OK THEN new board appears.
**6** AS A user I WANT TO handle tie GIVEN equal scores WHEN game ends THEN "Tie!" is shown.
**7** AS A user I WANT TO use 7×7 GIVEN current is 3×3 WHEN I select 7×7 THEN large grid fits perfectly.
**8** AS A user I WANT TO verify first player GIVEN new game WHEN first move THEN Red gets credit.
**9** AS A user I WANT TO ignore full cells GIVEN cell is 4 WHEN I click it THEN no change.
**10** AS A user I WANT TO close cleanly GIVEN window open WHEN I click X THEN app exits.
**11** AS A user I WANT TO play multiple games GIVEN one ended WHEN dialog closes THEN scores reset.
**12** AS A user I WANT TO score correctly GIVEN Blue fills last 3 cells WHEN game ends THEN Blue +3.
**13** AS A user I WANT TO test large board GIVEN 7×7 WHEN all filled THEN correct winner.
**14** AS A user I WANT TO avoid scrollbars GIVEN any size WHEN pack() called THEN all visible.

**15** AS A user I WANT TO auto-restart GIVEN winner dialog WHEN OK clicked THEN new board instantly.

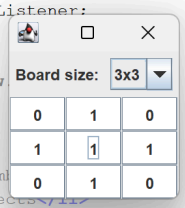
## Screenshots_

```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 */

package com.mycompany.fourgame;

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

/**
 * FourGame - the main window
 * <p>
 * Handles:
 * <ul>
 *   <li>size selection (JCom
 *   <li>grid of JButton objects</li>
 *   <li>event routing to Board</li>
 *   <li>game-over dialog and automatic restart</li>
 * </ul>
 */
public class FourGame extends JFrame implements ActionListener {
    private Board board;
```

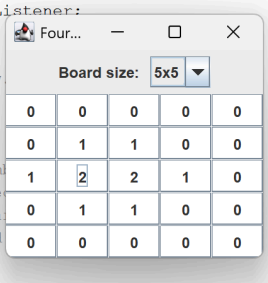



```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 */

package com.mycompany.fourgame;

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

/**
 * FourGame - the main window
 * <p>
 * Handles:
 * <ul>
 *   <li>size selection (JCom
 *   <li>grid of JButton obje
 *   <li>event routing to Boa
 *   <li>game-over dialog and
 * </ul>
 */
public class FourGame extends JFrame implements ActionListener {
    private Board board;
```

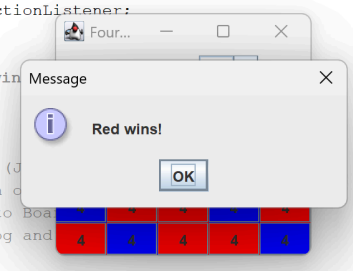
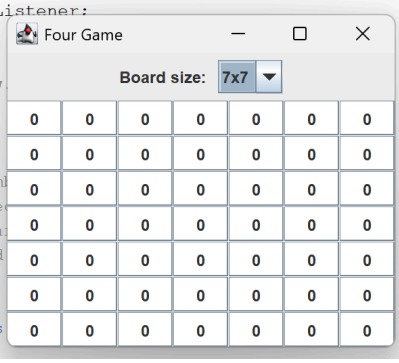



```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 */

package com.mycompany.fourgame;

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

/**
 * FourGame - the main win
 * <p>
 * Handles:
 * <ul>
 *   <li>size selection (J
 *   <li>grid of JButton c
 *   <li>event routing to Boa
 *   <li>game-over dialog and
 * </ul>
 */
public class FourGame extends JFrame implements ActionListener {
    private Board board;
```



```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 */

package com.mycompany.fourgame;

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

/**
 * FourGame - the main window
 * <p>
 * Handles:
 * <ul>
 *   <li>size selection (JCom
 *   <li>grid of JButton obje
 *   <li>event routing to Boa
 *   <li>game-over dialog and
 * </ul>
 */
public class FourGame extends
    private Board board;
```